Univerzitet u Sarajevu
Elektrotehnički fakultet
Predmet: Uvod u digitalnu obradu slike
Predmetni profesor: prof. dr Emir Sokić
Autor: Berina Biberović
Akademska godina: 2022/2023.

Tutorijal 5

Operacije nad slikama u frekventnom domenu. *Deblur*.

Ovaj tutorijal će da bude uputstvo za laboratorijsku vježbu 5 koja se bavi *deblur* efektom nad slikama, zbog toga što je primjećeno da laboratorijska vježba 5 predstavlja veliki problem studentima prilikom izrade.

Kod priložen u tutorijalu je zasnovan na kodu sa linka. Autor je morao napraviti izmjene kako bi se dobili željeni rezultati, zbog čega se korišteni kod nalazi ispod. Pretpostavka je da je *blur* koji je nastao na slici posljedica linearnog kretanja.

```
#include <string>
#include <opencv4/opencv2/imgproc.hpp>
#include <opencv4/opencv2/highgui.hpp>
#include <opencv4/opencv2/core/mat.hpp>
#include <opencv4/opencv2/imgcodecs.hpp>
#include <opencv4/opencv2/core/matx.hpp>


// Odabrati jednu od funkcija.
void calcPSF(cv::Mat& outputImg, cv::Size filterSize, int len, double theta) {  // Linear motion
    cv::Mat h(filterSize, CV_32F, cv::Scalar(0));
    cv::Point point(filterSize.width / 2, filterSize.height / 2);
    cv::ellipse(h, point, cv::Size(0, cvRound(float(len) / 2.0)), 90.0 - theta, 0, 360, cv::Scalar(255), cv::FILLED);
    cv::Scalar summa = sum(h);
    cv::namedWindow("FILTER", cv::WINDOW_NORMAL);
    imshow("FILTER", h);
    outputImg = h / summa[0];
}


// Pogledati https://docs.opencv.org/4.x/d8/d01/tutorial_discrete_fourier_transform.html
void fftshift(const cv::Mat& inputImg, cv::Mat& outputImg) {
    outputImg = inputImg.clone();
    int cx = outputImg.cols / 2;
    int cy = outputImg.rows / 2;
    cv::Mat q0(outputImg, cv::Rect(0, 0, cx, cy));
    cv::Mat q1(outputImg, cv::Rect(cx, 0, cx, cy));
    cv::Mat q2(outputImg, cv::Rect(0, cy, cx, cy));
    cv::Mat q3(outputImg, cv::Rect(cx, cy, cx, cy));
    cv::Mat tmp;
    q0.copyTo(tmp);
    q3.copyTo(q0);
    tmp.copyTo(q3);
    q1.copyTo(tmp);
    q2.copyTo(q1);
    tmp.copyTo(q2);
```

```
}

void filter2DFreq(const cv::Mat& inputImg, cv::Mat& outputImg, const cv::Mat& H) {
    cv::Mat planes[2] = { cv::Mat_<float>(inputImg.clone()), cv::Mat::zeros(inputImg.size(), CV_32F) };
    cv::Mat complexI;
    merge(planes, 2, complexI);
    dft(complexI, complexI, cv::DFT_SCALE);
    cv::Mat planesH[2] = { cv::Mat_<float>(H.clone()), cv::Mat::zeros(H.size(), CV_32F) };
    cv::Mat complexH;
    cv::merge(planesH, 2, complexH);
    cv::Mat complexIH;
    cv::mulSpectrums(complexI, complexH, complexIH, 0);
    cv::idft(complexIH, complexIH);
    cv::split(complexIH, planes);
    outputImg = planes[0];
}

void calcWnrFilter(const cv::Mat& input_h_PSF, cv::Mat& output_G, double nsr) {
    cv::Mat h_PSF_shifted;
    fftshift(input_h_PSF, h_PSF_shifted);
    cv::Mat planes[2] = { cv::Mat_<float>(h_PSF_shifted.clone()), cv::Mat::zeros(h_PSF_shifted.size(), CV_32F) };
    cv::Mat complexI;
    cv::merge(planes, 2, complexI);
    cv::dft(complexI, complexI);
    cv::split(complexI, planes);
    cv::Mat denom;
    pow(abs(planes[0]), 2, denom);
    denom += nsr;
    cv::divide(planes[0], denom, output_G);
};

void edgetaper(const cv::Mat& inputImg, cv::Mat& outputImg, double gamma, double beta) {
    int Nx = inputImg.cols;
    int Ny = inputImg.rows;
    cv::Mat w1(1, Nx, CV_32F, cv::Scalar(0));
    cv::Mat w2(Ny, 1, CV_32F, cv::Scalar(0));
    float* p1 = w1.ptr<float>(0);
    float* p2 = w2.ptr<float>(0);
    float dx = float(2.0 * CV_PI / Nx);
    float x = float(-CV_PI);
    for (int i = 0; i < Nx; i++) {
        p1[i] = float(0.5 * (tanh((x + gamma / 2) / beta) - tanh((x - gamma / 2) / beta)));
        x += dx;
    }
    float dy = float(2.0 * CV_PI / Ny);
    float y = float(-CV_PI);
    for (int i = 0; i < Ny; i++) {
        p2[i] = float(0.5 * (tanh((y + gamma / 2) / beta) - tanh((y - gamma / 2) / beta)));
        y += dy;
    }
    cv::Mat w = w2 * w1;
    cv::multiply(inputImg, w, outputImg);
```

```
}

int main(){
    std::string ime = "slika.jpeg";
    cv::Mat slika;
    slika = imread("../" + ime, cv::IMREAD_GRAYSCALE);

    cv::Mat rezultat;

    // Srezati sliku na paran broj redova i kolona
    cv::Rect roi = cv::Rect(0, 0, slika.cols & -2, slika.rows & -2);

    // Racunanje Wienerovog filtera
    cv::Mat Hw, h;

    cv::namedWindow("Trackbars", cv::WINDOW_NORMAL);
    cv::namedWindow("Izlaz", cv::WINDOW_NORMAL);

    cv::createTrackbar("t1", "Trackbars", nullptr, 500);
    cv::createTrackbar("t2", "Trackbars", nullptr, 180);
    cv::createTrackbar("t3", "Trackbars", nullptr, 1000);

    char keyPress;
    cv::Mat pomocna;
    while(true) {

        int t1 = cv::getTrackbarPos("t1", "Trackbars");
        int t2 = cv::getTrackbarPos("t2", "Trackbars");
        int t3 = cv::getTrackbarPos("t3", "Trackbars");

        calcPSF(h, roi.size(), t1, t2);
        calcWnrFilter(h, Hw, 1.0 / double(t3));

        slika.convertTo(slika, CV_32F);

        edgetaper(slika, pomocna, 10, 10);

        // Filtriranje
        filter2DFreq(pomocna(roi), rezultat, Hw);

        // Prikazivanje filtrirane slike
        rezultat.convertTo(rezultat, CV_8U);
        cv::normalize(rezultat, rezultat, 0, 255, cv::NORM_MINMAX);
        cv::imshow("Izlaz", rezultat);

        if(cv::waitKey() == 'q') break;
    }

    cv::imwrite("../Rezultat.jpeg", rezultat);
    return 0;
}
```

U gornjem kodu možemo primijetiti da smo napravili 5 *user defined* funkcija: `calcPSF`, `fftshift`, `filter2DFreq`, `calcWnrFilter` i `edgetaper`. Sve ove funkcije će biti objašnjene kako budemo prolazili kroz `main` funkciju.

Unutar `main` funkcije učitavamo *grayscale* sliku nad kojom želimo da maknemo mućenje (eng. *blur*). Nakon toga korištenjem konstruktora klase `cv::Rect` ćemo srezati sliku na paran broj redova i kolona - rezultat toga stavljamo u objekat istog tipa. Naravno, imenujemo odgovarajuće instance klase `cv::Mat` koje će nam kasnije trebati - `rezultat`, `Hw`, `pomocna` i `h`.

Onda pravimo tri *trackbar*-ove uz pomoć kojih ćemo sklanjati uticaj mućenja sa slike, što će detaljno biti u nastavku objašnjeno. Ulaskom u `while` petlju, prvo uzimamo vrijednosti tri varijable `t1`, `t2` i `t3` za koje smo napravili prethodno *trackbar*-ove.

Pozivamo onda funkciju `calcPSF` kojoj prosljeđujemo `h`, veličinu varijable `roi`, `t1` i `t2`. Ova funkcija služi da se uz pomoć *linear motion*-a dobije slika kretanja. Detaljno možete o ovome pročitati na linku.

Nakon toga pozivamo funkciju `calcWnrFilter` koja računa Wienerov filter. Prosljeđujemo joj varijable `h`, `Hw`, `1.0 / t3`. Ključan aspekt za primjenu Wienerovog filtera za pokušaj uklanjanja šuma sa slike je precizno definisanje kretanja objekata na slici. U ovu svrhu koristimo *point spread* funkciju, koja definiše kako se jedna tačka pomjerila za vrijeme otvora blende kamere pri pravljenju fotografije. Pri pokretanju programa potrebno je uskladiti ovu funkciju sa kretanjem u prostoru slike korištenjem parametara `len` i `theta` (koji odgovaraju količini kretanja jednog piksela u području od interesa i ugao u ravni slike). Pored ovoga porebno je odrediti SNR radi primjene Wienerovog filtra.

Ključna pretpostavka je da je kretanje svake tačke u ravni slike identično. Naravno, ovo nije valjana pretpostavka uvijek, ali se može pretpostaviti da je tačna lokalno, čime se omogućuje *deblur* efekat objekta ili dijela slike od interesa. Kako se u idealnom slučaju šum uslijed kretanja manifestuje kao konvolucija slike sa binarnom slikom koja sadrži liniju koja predstavlja kretanje jedne svake tačke u slici tako je *deblur* zapravo problem dekonvolucije. Wienerova dekonvolucija koristi upravo Wienerov filter kako bi se uklonio šum.

Vrijedno spomena je da njegova direktna primjena dolazi do problema pri ivici slike, pošto je potrebna informacija o slici van okvira slike, što dovodi do tzv. *ringing* efekta. Uslijed digitalne prirode slike dolazi do nepovratnog gubitka informacija zbog čega je nerazumno očekivati od bilo kakve metode da uradi savršen *debluring*, no *debluring* metode mogu itekako pomoći u povratku dijela kvaliteta slike. Da bismo što više mogli umanjiti *ringing* efekat, korištena je za to funkcija `edgetaper` koja prima u prethodnoj liniji konvertovanu sliku u tip `CV_32F`, spašava rezultat funkcije u varijablu `pomocna` i prima dva parametra `gamma` i `beta` koji služe za transformaciju piksela na rubu slike.

Nakon toga se poziva funkcija `filter2DFreq` koja prima ulaznu sliku `pomocna(roi)`, vraća izlaznu sliku `rezultat` i prima matricu `Hw` koju smo dobili iz funkcije `calcWnrFilter`. `rezultat` se onda konvertuje u tip `CV_8U`, normalizira se i na ekranu pokazujemo izgled slike nakon primjenjivanja svih ovih filtera na njega.

Za funkciju `fftshift` koja radi *shift*-anje u 2D-u, možete detaljnije pročitati na linku.

Originalna slika koja je ubačena u filter je prikazana na slici 1. Rezultat nakon filtriranja koji je prikazan na slici 2 je dobiven za parametre `t1 = 18`, `t2 = 11` i `t3 = 48` za datu sliku.



Slika 1: Originalna slika



Slika 2: Rezultat nakon *debluring*-a

Napomene

Korištena tema za prikaz C++ koda na tutorijalima je `solarized`[1].

Literatura

[1] Ethan Schoonover. *Solarized*. `http://ethanschoonover.com/solarized`. Pristup stranici: 20. maj 2023. 2011.